3D nesting benchmark

Ten packing algorithms, six part categories, one Monte Carlo test bench

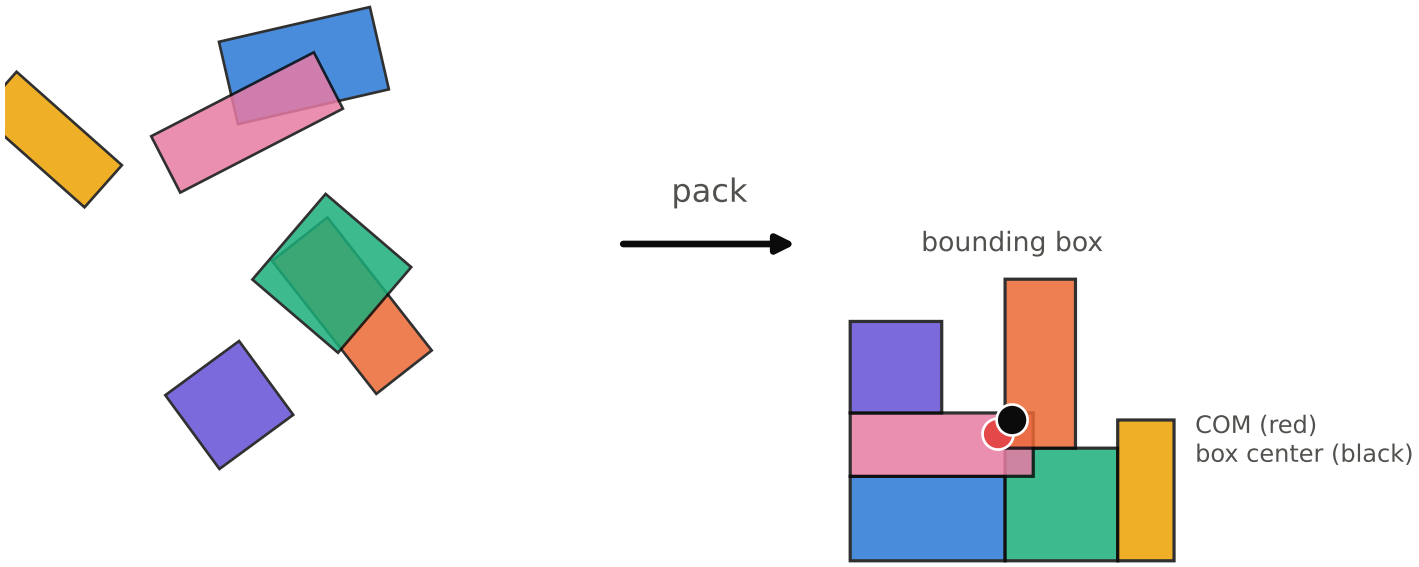
Text to Spaceship · UC Berkeley MEng capstone with NASA Goddard

Dadmehr Daniel Ghasemfar · November 5, 2026

Code, datasets and results: nesting_bench/ (python viewer.py to explore, python benchmark.py to reproduce)

The problem

Given k parts, find positions and orientations so they take the least space without touching



Two modes

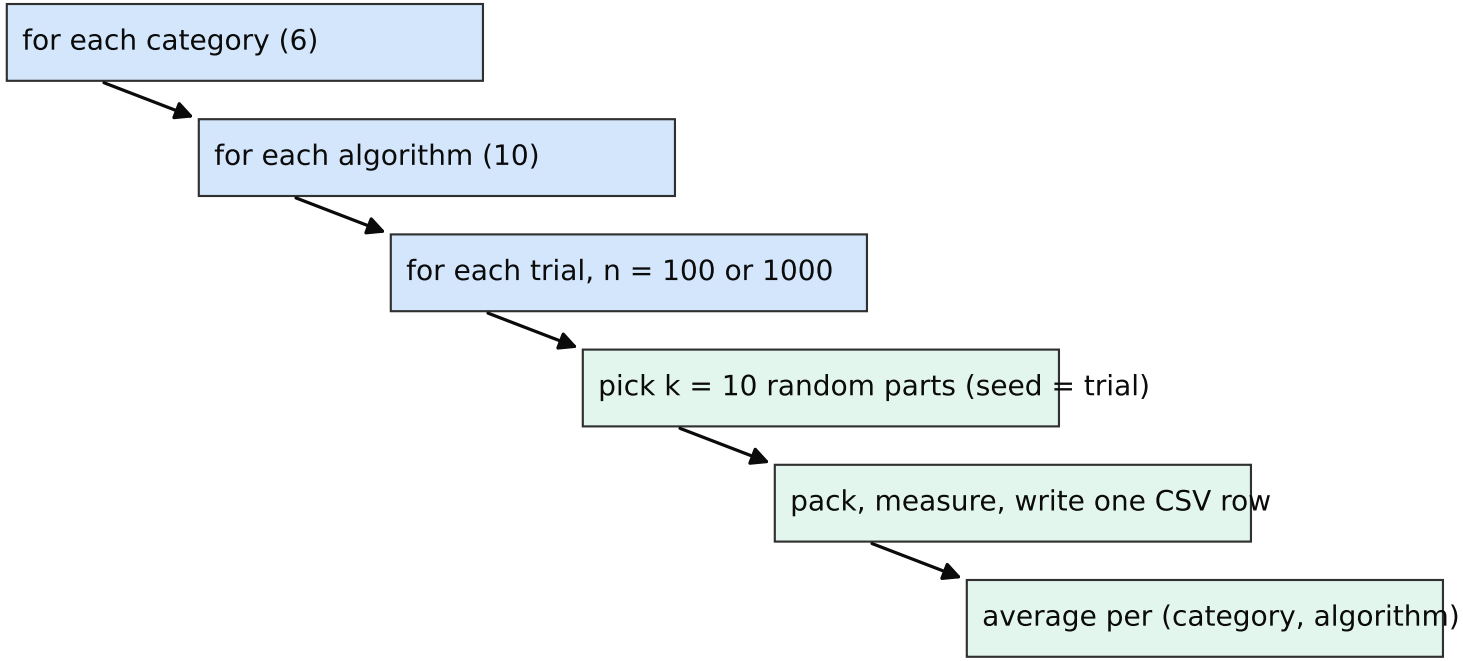
- open: minimize the bounding box
- container: fill a fixed box (e.g. 100 mm cube)

What we measure per packing

- density = parts volume / box volume
- aspect ratio = longest / shortest side
- COM to box center, % of diagonal
- inertia tensor and principal moments
- placement order (assemblability later)
- minimum gap, must be ≥ t = 0.5 mm
- valid = no overlap (voxel check, exact spot checks)

The test procedure

Monte Carlo: same random part sets for every algorithm, then average



score = mean density, with invalid (overlapping) packings counted as 0

Fairness

trial t uses seed t for the part draw,

so every algorithm packs the same sets

Randomized algorithms

3 restarts, best valid kept

Cost

6 × 10 × 100 = 6,000 packings per run

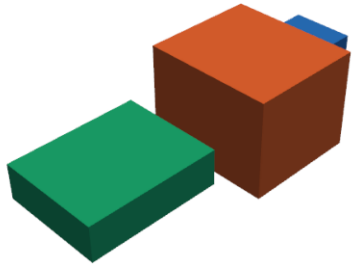
n = 1000 for the six original methods:

36,000 packings

The parts: six categories, 123 watertight STLs

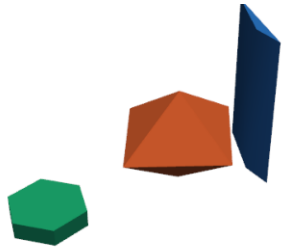
Longest side 15 to 45 mm, centered; every part checked watertight, one body, positive volume

cuboids



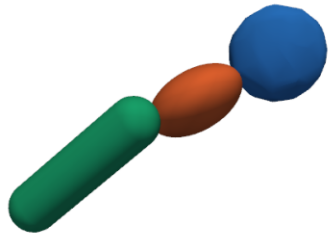
20 random boxes, 8 to 45 mm

simple convex



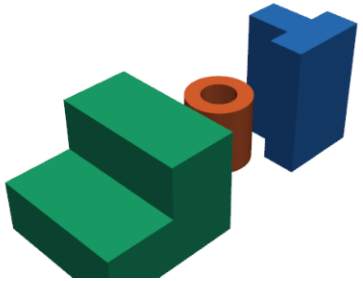
20: prisms, pyramids, cylinders, cones, wedges

complex convex



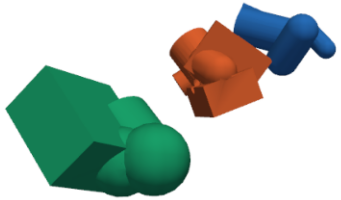
20: random hulls, ellipsoids, capsules, clipped boxes

simple concave



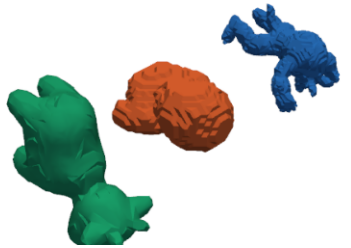
20: L, T, U, I, cross, tube, step, plate with hole

complex concave



20: unions of 3 to 7 primitives, 0 to 2 holes

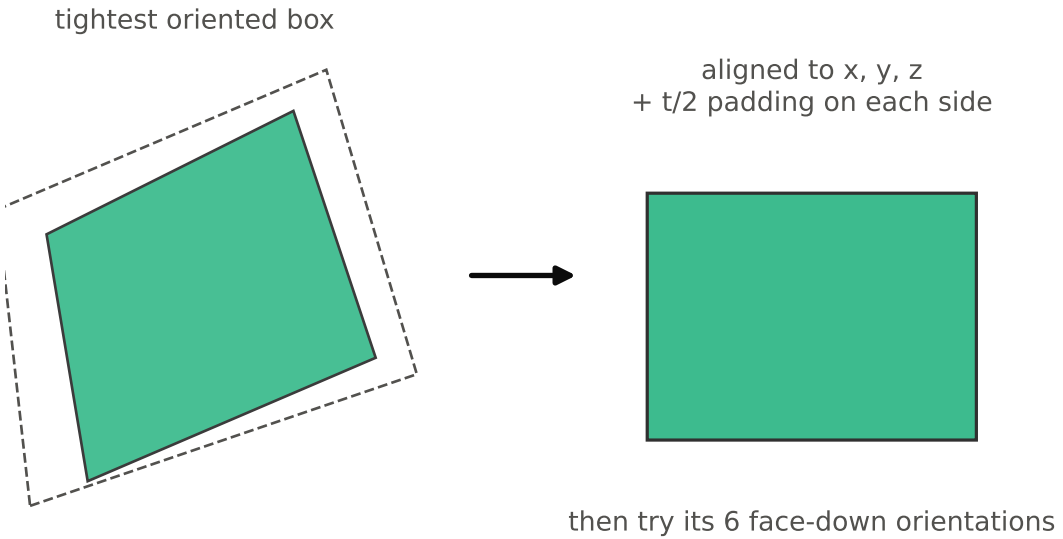
real shapes



23: bunny, dragon, teapot, armadillo ... (remeshed)

Rules every algorithm follows

So the comparison tests the placement logic, not the bookkeeping



Orientation

first align the part's minimum-volume oriented box to the axes, then try the 6 ways to set that box down (24 optional)

random methods use any angle

Clearance t = 0.5 mm

boxes are padded by t, voxels grown by t/2, so no two parts come closer than t

Order

largest box first, unless the algorithm searches over the order itself

Ten algorithms in three families

Boxes are fast but blind to shape; shapes can interlock; search improves any of them

baseline

random rejection

- random rejection

box based

each part is its padded box

- shelf packing
- box bottom left
- box extreme points
- force based
- force based v2 (new)

shape based

the real shape on a grid

- voxel bottom left
- heightfield drop (new)

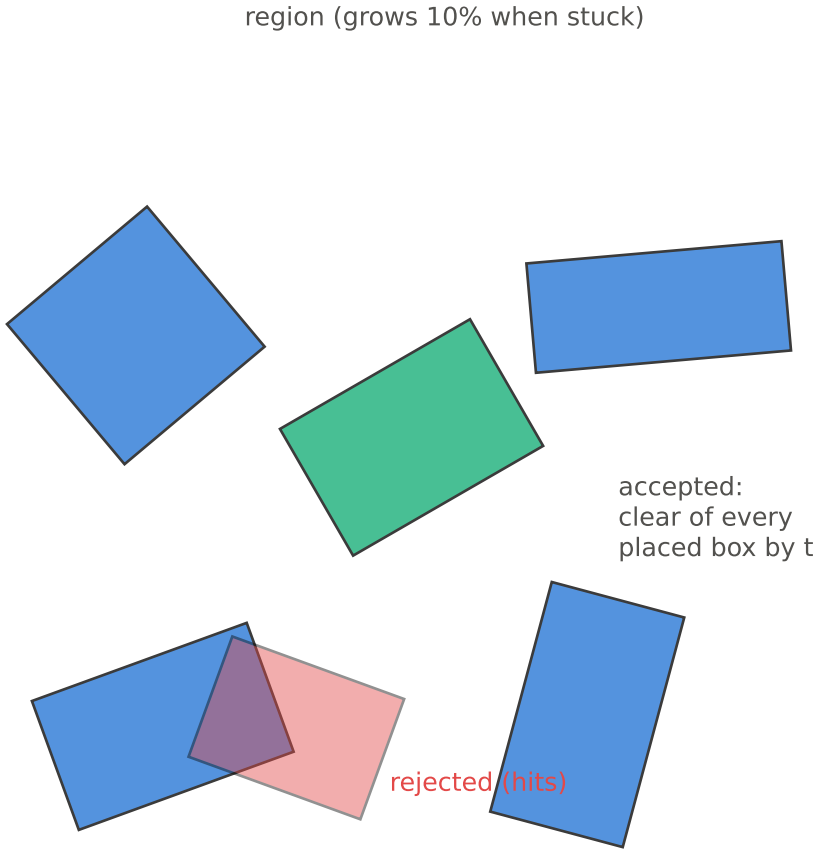
search based

optimize order + orientation

- simulated annealing (new)
- genetic algorithm (new)

random rejection

baseline family



How

each part: random angle, random position inside a cube

reject if its box comes within t of any placed box

300 tries, then grow the cube 10% and try again

3 restarts, best kept

Strength

the floor every real method must beat

any orientation, no assumptions

Weakness

no packing logic at all

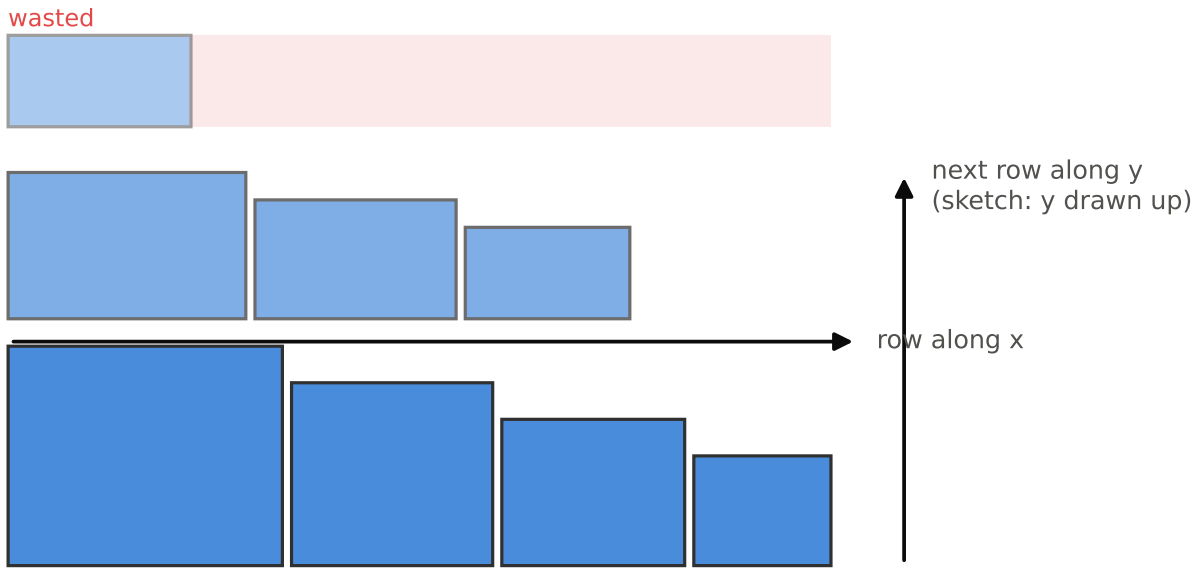
density 0.09 to 0.19; gaps 2 to 3 mm

n = 100, k = 10 density: cuboids 0.19 simple concave 0.12 real shapes 0.09 ·

shelf packing

box family

tallest first, laid flat (shortest side up)
row height = tallest part in the row
new layer in z when the floor is full



How

lay every part flat: shortest side up, longest along x

tallest first; fill a row along x, then the next row in y,

then a new layer in z when the square floor is full

Strength

fastest method (0.03 s per packing)

predictable layers; good for flat parts

Weakness

a tall part sets the whole layer height

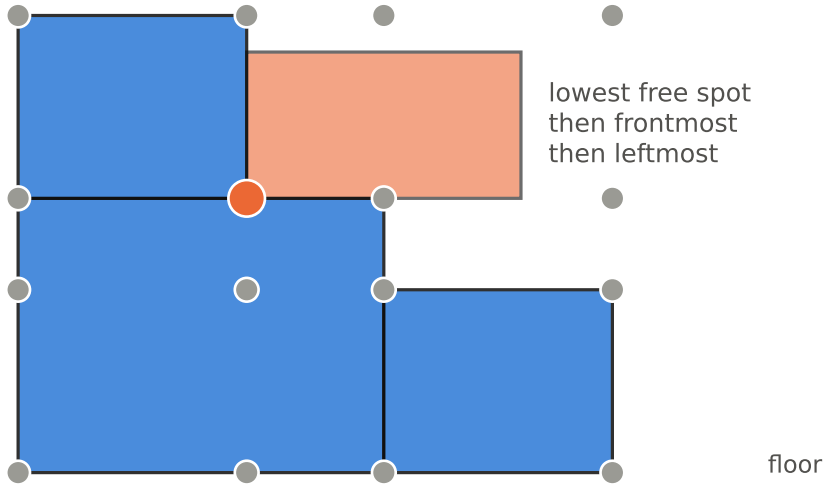
space above shorter parts is wasted

n = 100, k = 10 density: cuboids 0.49 simple concave 0.32 real shapes 0.14 ·

box bottom left

box family

candidates = every combination of
placed box edges (grey dots)
square floor ≈ cube root of total volume



How

largest box first

candidate corners: every combination of placed box

edges (x, y, z maxima and 0) on a square floor

take the lowest, then frontmost, then leftmost free

corner over the 6 orientations

Strength

near-cubic packings (aspect 1.1 to 1.3)

simple, deterministic, 0.05 s

Weakness

fixed floor size limits the layout

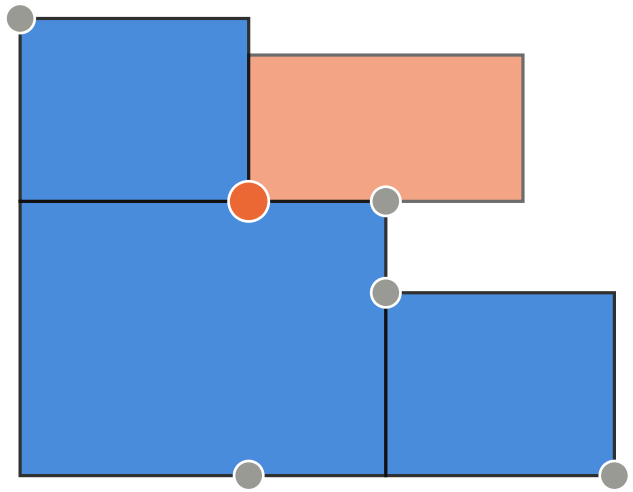
boxes only: cannot use a concave part's notch

n = 100, k = 10 density: cuboids 0.62 simple concave 0.39 real shapes 0.15 ·

box extreme points

box family

candidates = only the 3 corners each placed box exposes (+x, +y, +z) pick spot + orientation that keeps the overall box (solid line) smallest



fewer candidates than bottom-left, better objective

How

largest box first

candidates: only the 3 corners each placed box exposes

choose the corner and orientation that keep the overall

bounding box smallest (container mode: lowest corner)

Strength

densest box method on cuboids (0.78)

few candidates, so fast (0.04 s)

Weakness

greedy volume objective builds slabs

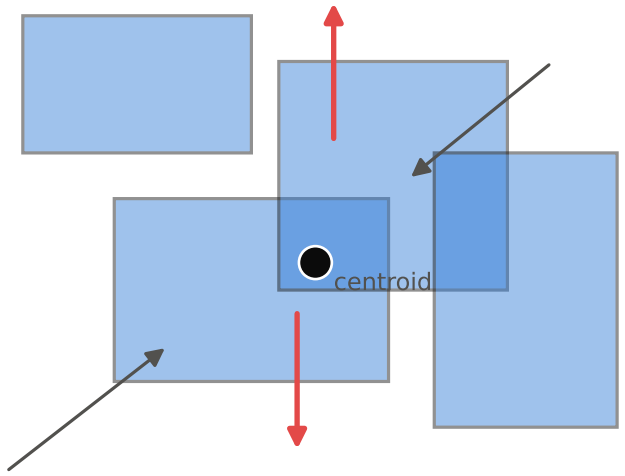
(aspect 4 to 6); boxes only

n = 100, k = 10 density: cuboids 0.77 simple concave 0.44 real shapes 0.20 ·

force based

box family

repeat 40×: squeeze all boxes 10% toward the centroid (grey)
then push each overlapping pair apart along its shallowest
overlap (red), up to 60 passes; keep the tightest overlap-free state



no placement order:
all parts move at once

How

every box starts at a random spot, overlapping

repeat 40×: squeeze 10% toward the centroid, then push

overlapping pairs apart along the shallowest overlap

keep the tightest overlap-free state (3 restarts)

Strength

no placement order; all parts move at once

constraints can be added as extra forces

Weakness

isotropic squeeze forms long rows (Izz/Ixx ≈ 10)

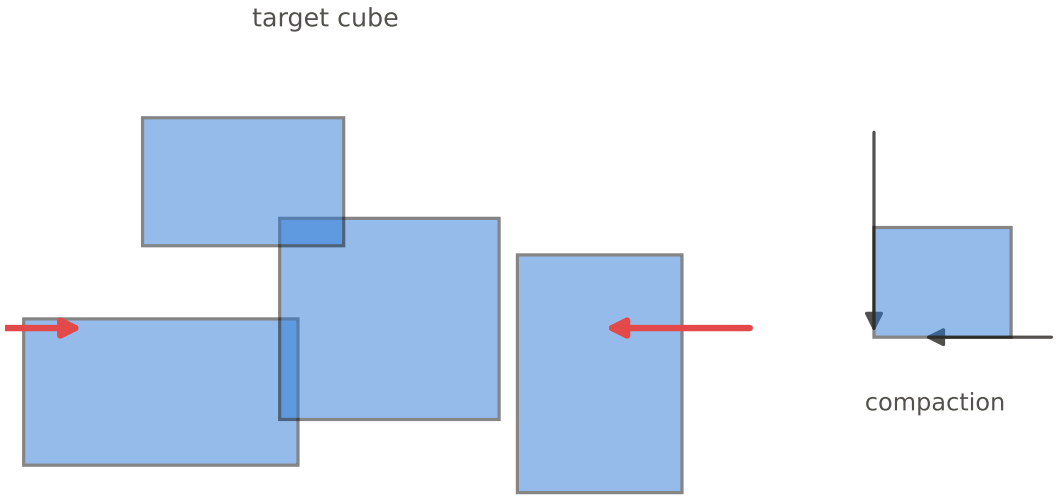
barely above random on concave parts

n = 100, k = 10 density: cuboids 0.45 simple concave 0.30 real shapes 0.11 ·

force based v2

box family · new today

1 anisotropic squeeze: an axis wider than the target is pulled in harder (big red arrows), so rows cannot form
2 compaction: every box slides toward the origin along z, y, x as far as the others allow
3 each box re-picks the orientation that overlaps least



How

as force_based, plus:

anisotropic squeeze toward a target cube

compaction: slide each box to the origin along z, y, x

re-pick each box's orientation to the one that

overlaps least where it sits

Strength

density 10 to 25% above v1, rows gone (Izz/Ixx 3 to 6 vs 8 to 12)

keeps the order-free, force-based structure

Weakness

still boxes, still below the greedy methods

still elongated (aspect 2.5 to 4); 0.4 s per packing

n = 100, k = 10 density: cuboids 0.51 simple concave 0.33 real shapes 0.13 ·

voxel bottom left

shape family

parts voxelized at 2 mm in their real shape, grown by t/2; the orange part sits inside the L's notch (boxes could not do this)



FFT convolution of grid ⊗ part = overlap count at every position; zero = free spot

keep the free spot that keeps the box smallest

How

each part voxelized at 2 mm in its real shape, grown by t/2

for each orientation, an FFT convolution of the grid with

the part gives the overlap count at every position at once

keep the free position that keeps the box smallest

Strength

concave parts can interlock

best on real shapes and simple convex

Weakness

2 mm grid + 0.5 mm clearance eat most of the

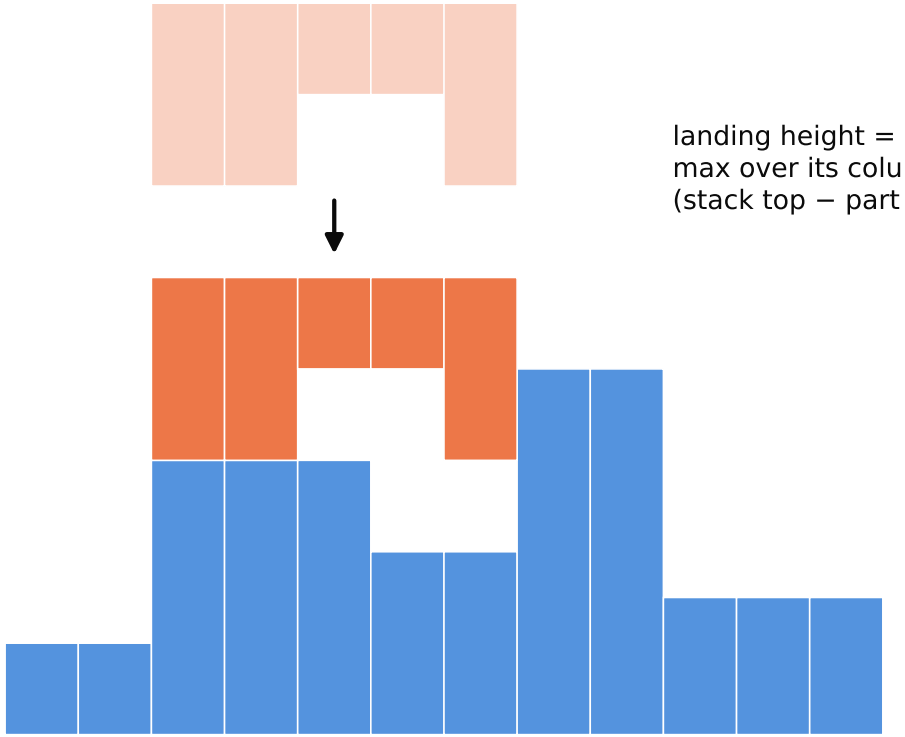
interlock gain; 5 to 10× slower than box methods

n = 100, k = 10 density: cuboids 0.74 simple concave 0.45 real shapes 0.22 ·

heightfield drop

shape family · new today

side view: the stack is a height map (blue tops); the part has a bottom profile and a top profile per column (orange); it rests on its first two columns



landing height =
max over its columns of
(stack top − part bottom)

tried at every (x, y)
and every orientation;
keep the spot that
keeps the box smallest

How

Huai-Yi's method, rebuilt on the benchmark's grid

the stack is a 2D height map; each part has a bottom

and a top profile per column

drop at every (x, y) and orientation: landing height =

max over its columns of (stack top − part bottom)

keep the landing that keeps the box smallest

Strength

overlap-free by construction (exact check: 0 mm³)

no 3D search: 0.04 to 0.08 s per packing, 5× faster than voxel

within 0.02 of voxel on every category

Weakness

a column is solid from bottom to top, so nothing

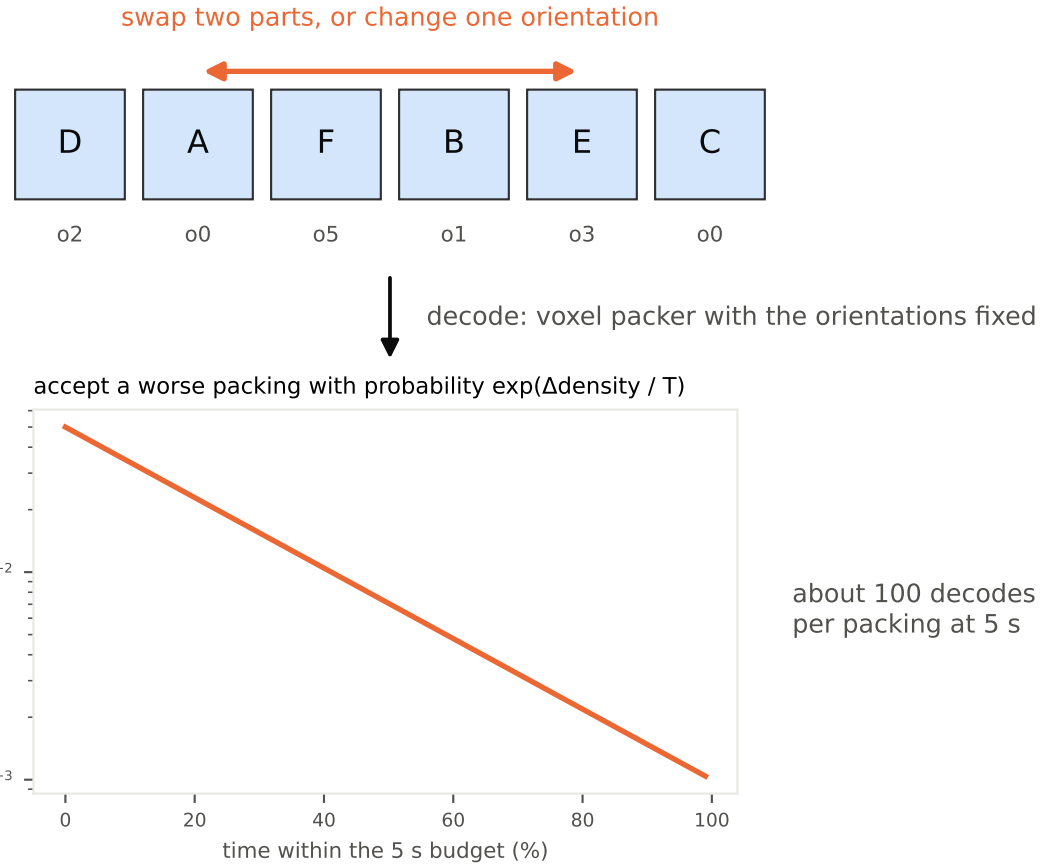
slides under an overhang (a sideways tube is a block)

greedy smallest-box choice builds towers (aspect 4 to 6)

n = 100, k = 10 density: cuboids 0.71 simple concave 0.43 real shapes 0.20 ·

simulated annealing

search family · new today



about 100 decodes per packing at 5 s

How

the decision = placement order + one orientation per part

the voxel packer (orientations fixed) decodes it to a packing

each step: swap two parts or re-orient one; accept if density rises, else with probability exp(Δ / T), T: 0.05 → 0.001

runs for a fixed 5 s budget (~100 decodes)

Strength

+0.01 to +0.03 density over voxel bottom-left on every category

most compact boxes (aspect 2 to 4) and lowest COM offset (3% of the diagonal) of any method

Weakness

5 s per packing; the result depends on the budget

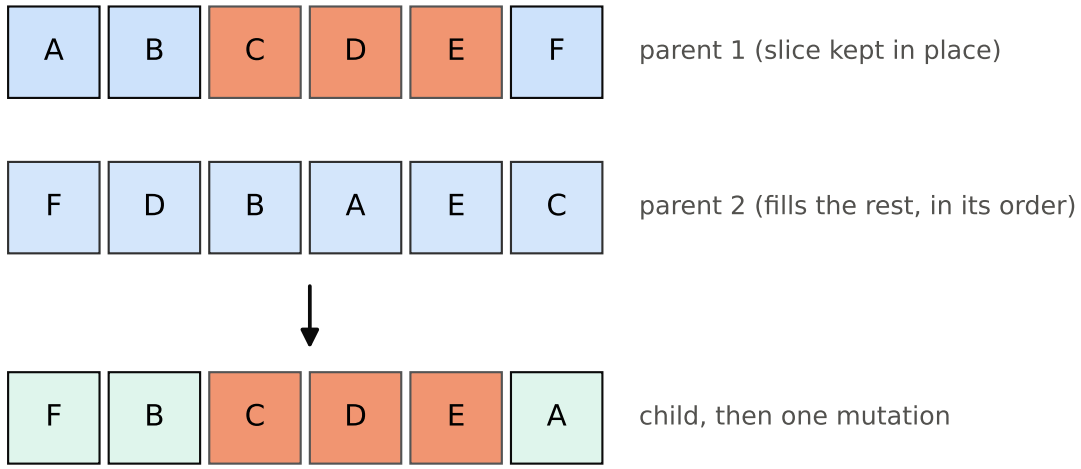
and on the CPU, so runs are not bit-reproducible

100 decodes is a small search for 10! orders

n = 100, k = 10 density: cuboids 0.75 simple concave 0.47 real shapes 0.23 ·

genetic algorithm

search family · new today



population 12; each generation keeps the best 6 and breeds 6 children
orientations come from either parent per part
decode and score every child (voxel packer); run for 5 s

How

same decision and decoder as annealing

population of 12; keep the best 6 each generation

children: order crossover (a slice of parent 1 stays in place, parent 2 fills the rest in its order), orientations from either parent, then one mutation; 5 s budget

Strength

best density on 5 of 6 categories (tied with extreme points on cuboids); parallel-friendly (children are independent)

Weakness

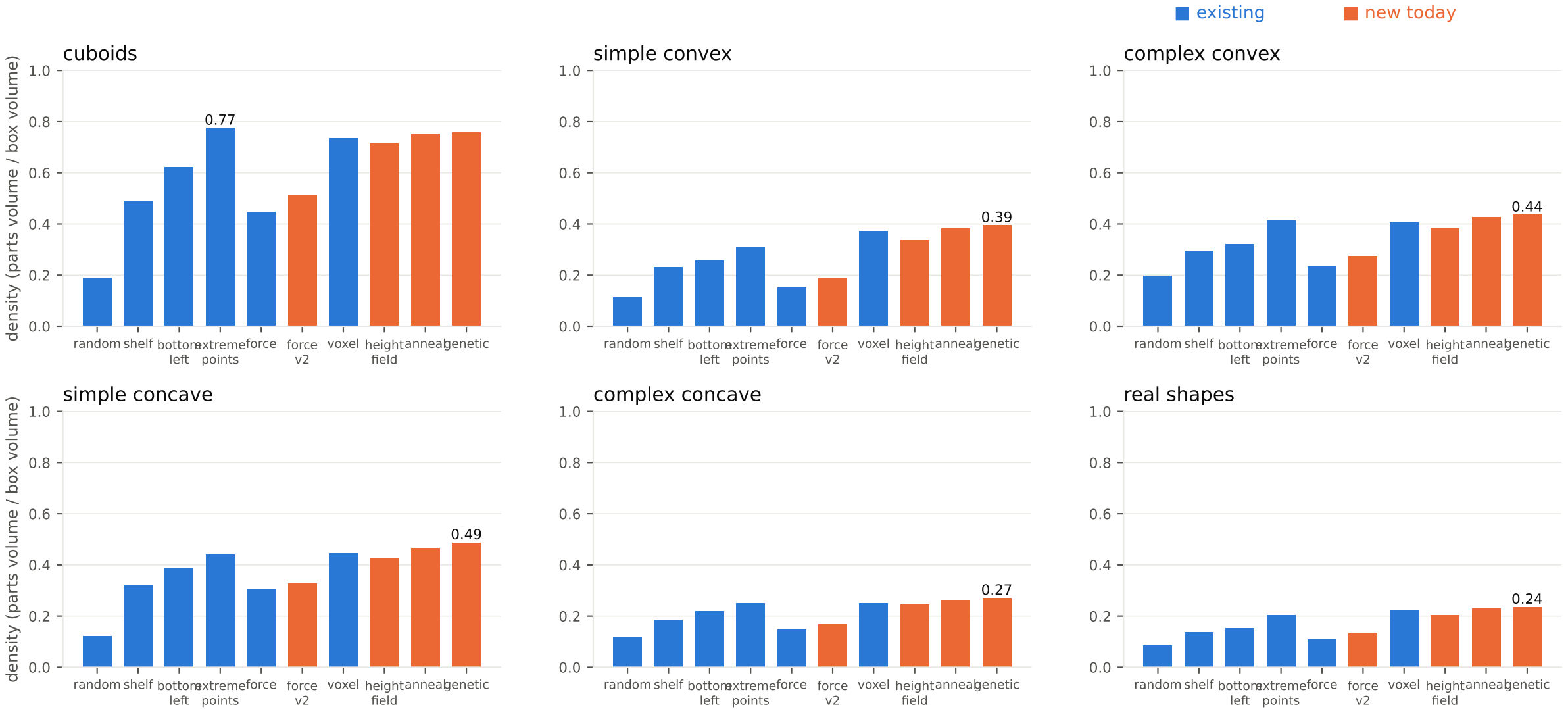
same 5 s cost; only +0.01 over annealing

same non-reproducibility (time budget)

n = 100, k = 10 density: cuboids 0.76 simple concave 0.49 real shapes 0.24 ·

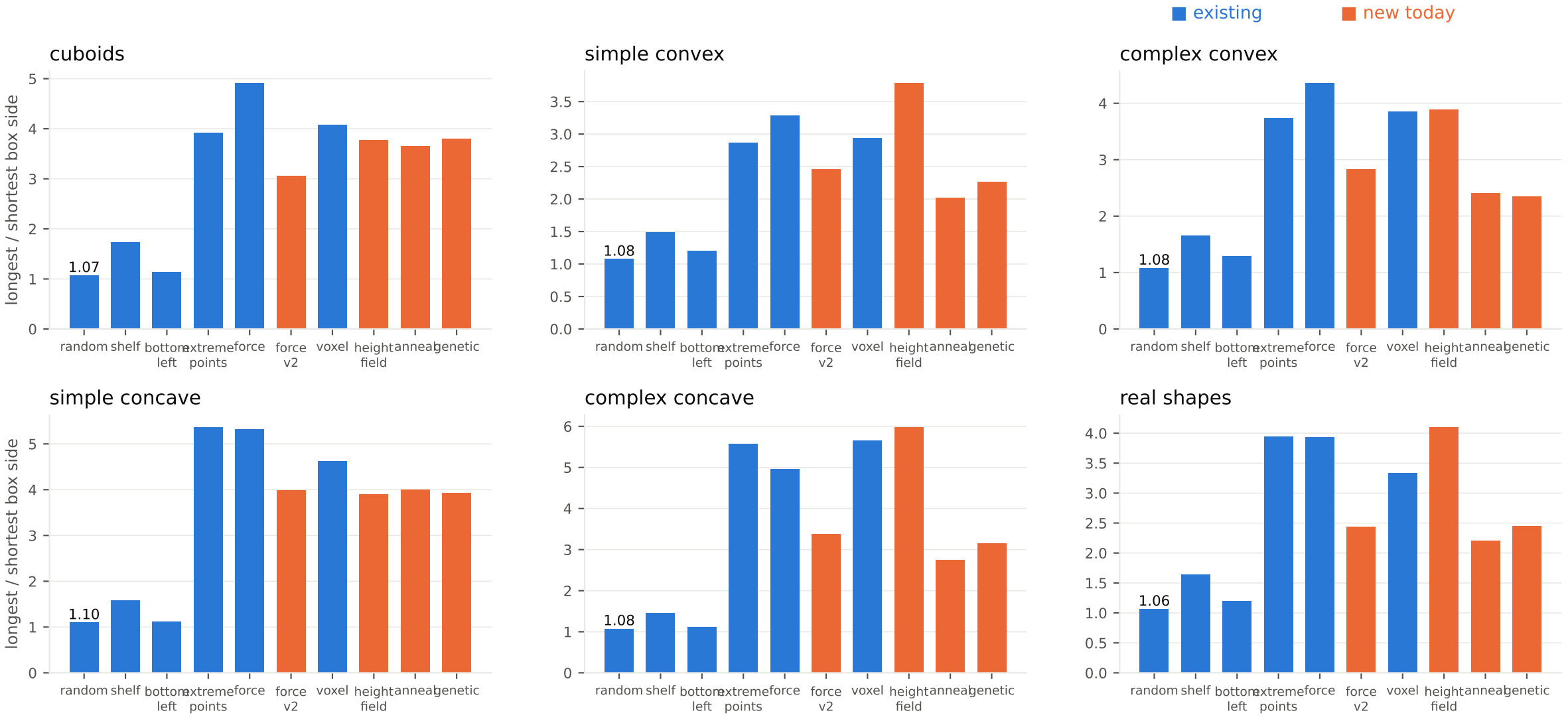
Results: density by category

n = 100 trials, k = 10 parts, open mode, t = 0.5 mm; invalid packings count as 0



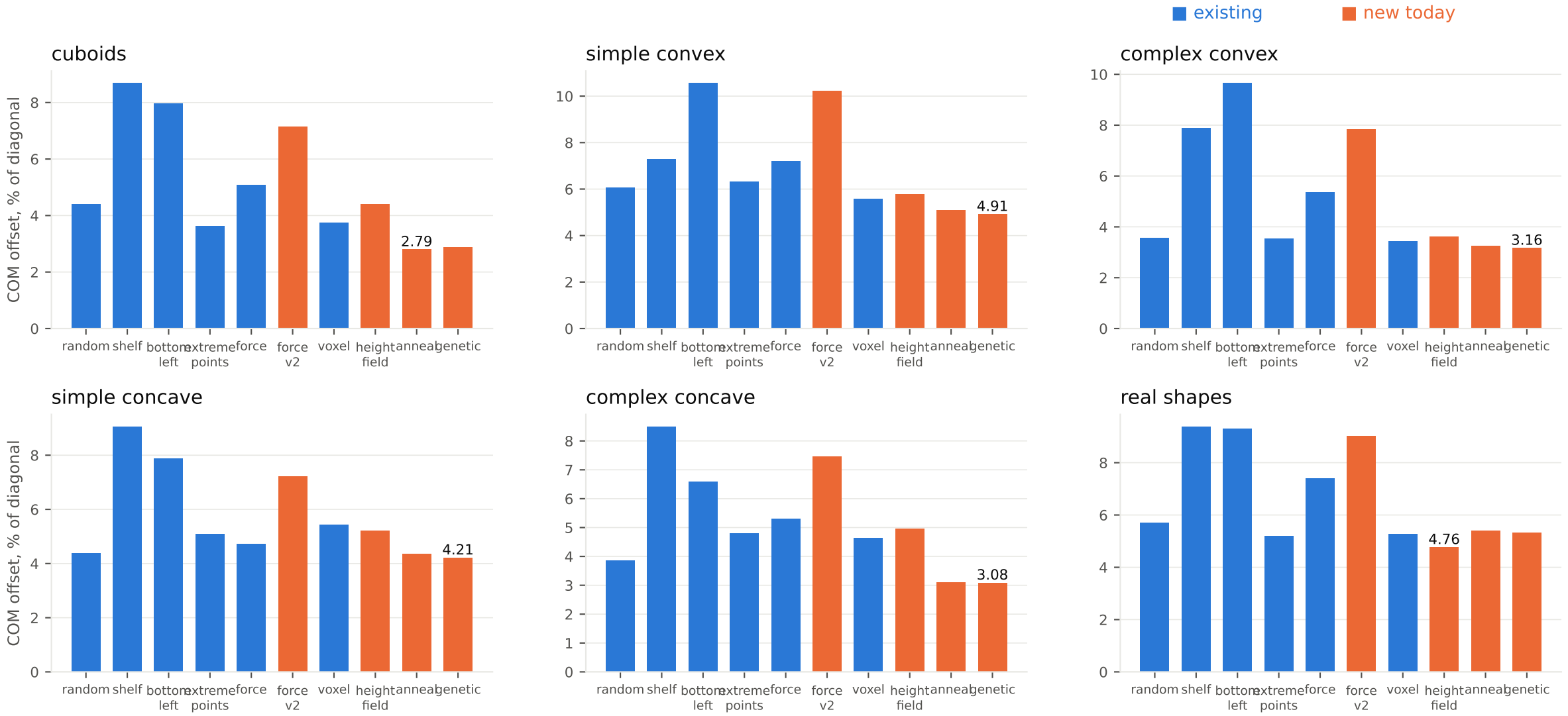
Results: aspect ratio (lower = more compact)

the densest packers make slabs; bottom-left and force v2 stay near-cubic



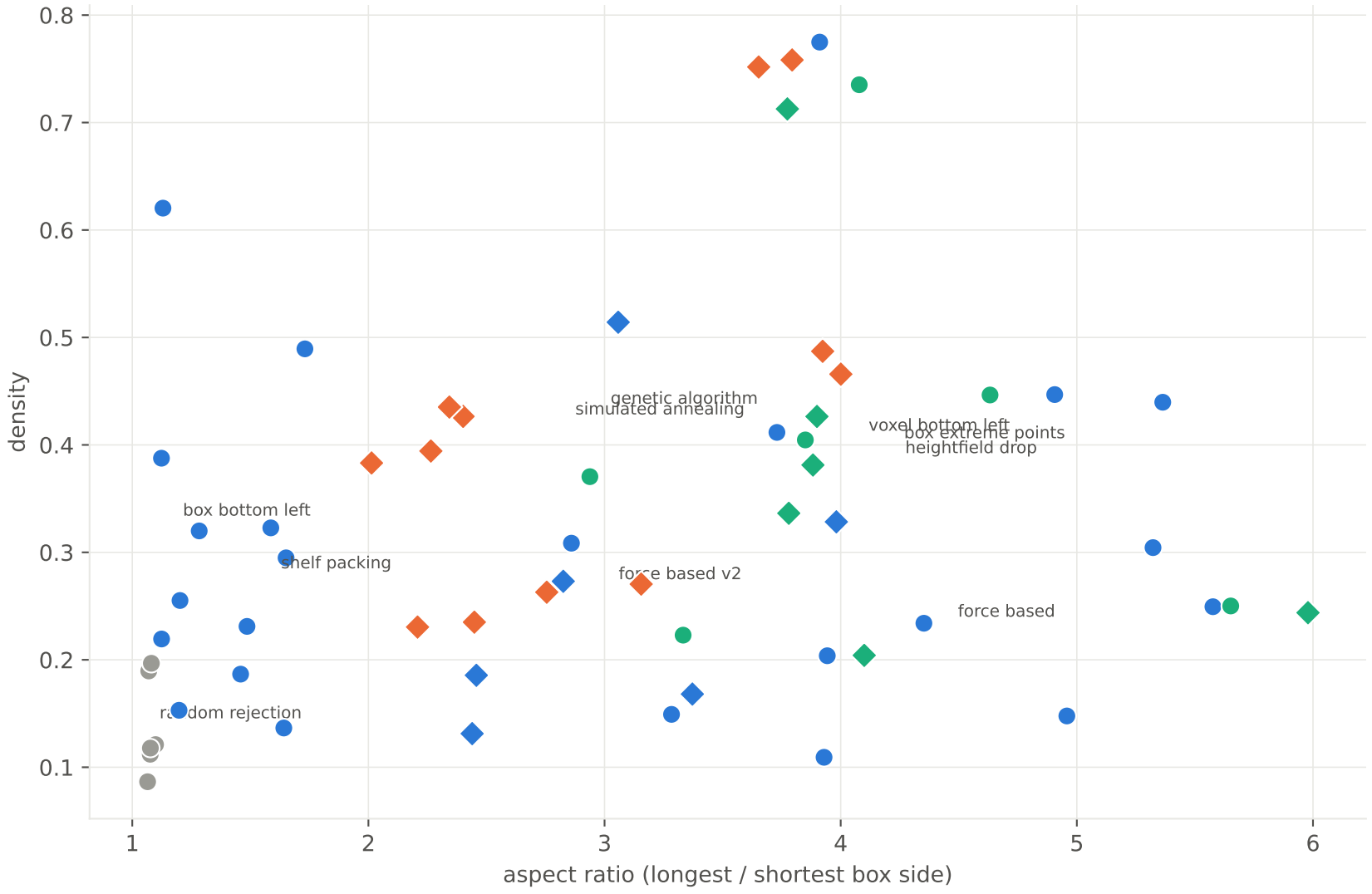
Results: center of mass offset (lower = better balanced)

distance from COM to box center as % of the box diagonal; no algorithm optimizes this yet



The trade-off: density costs shape

each point is one algorithm on one category (n = 100); up-left is the goal



color = family

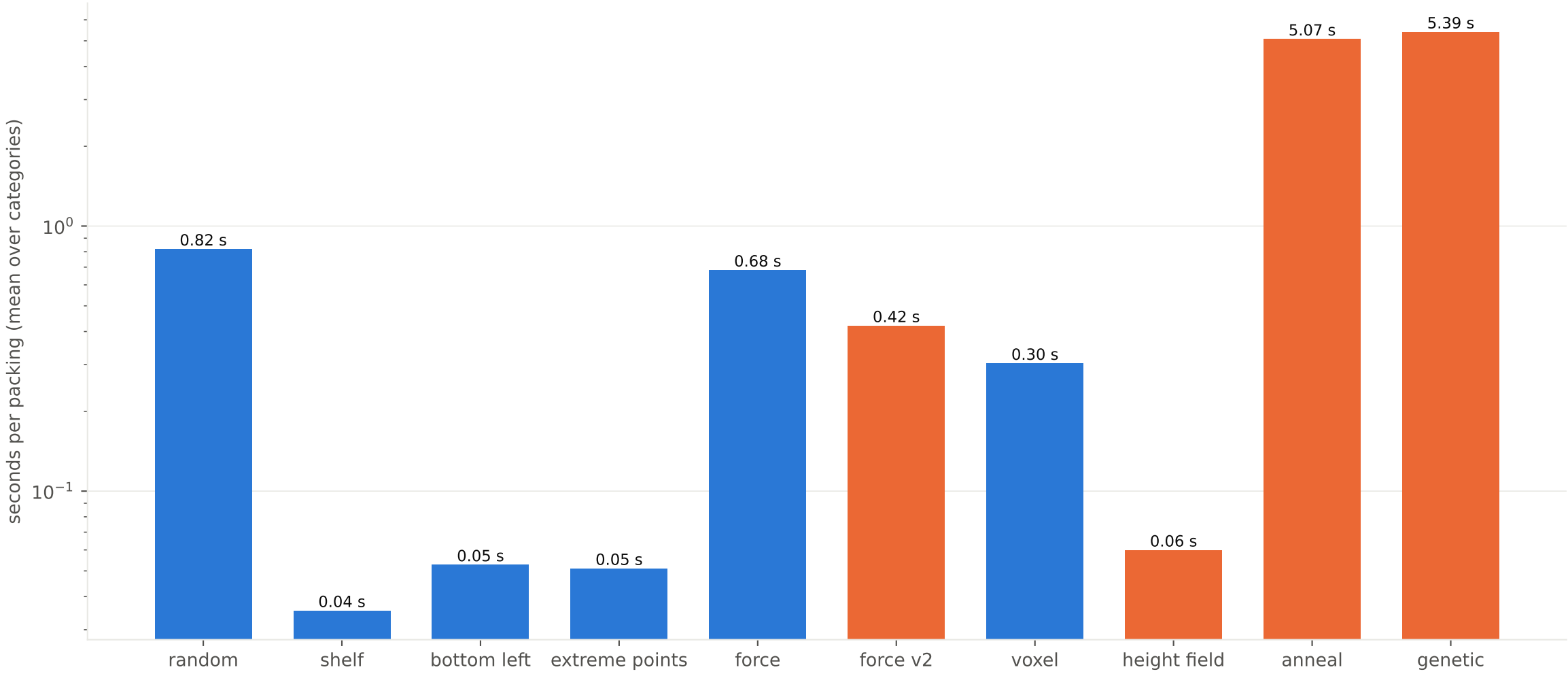
- grey baseline
- blue box based
- aqua shape based
- orange search based

diamond = new today

Greedy 'smallest box' choices stack parts into towers and slabs. A satellite wants a compact envelope, so the objective needs an aspect or target-box term.

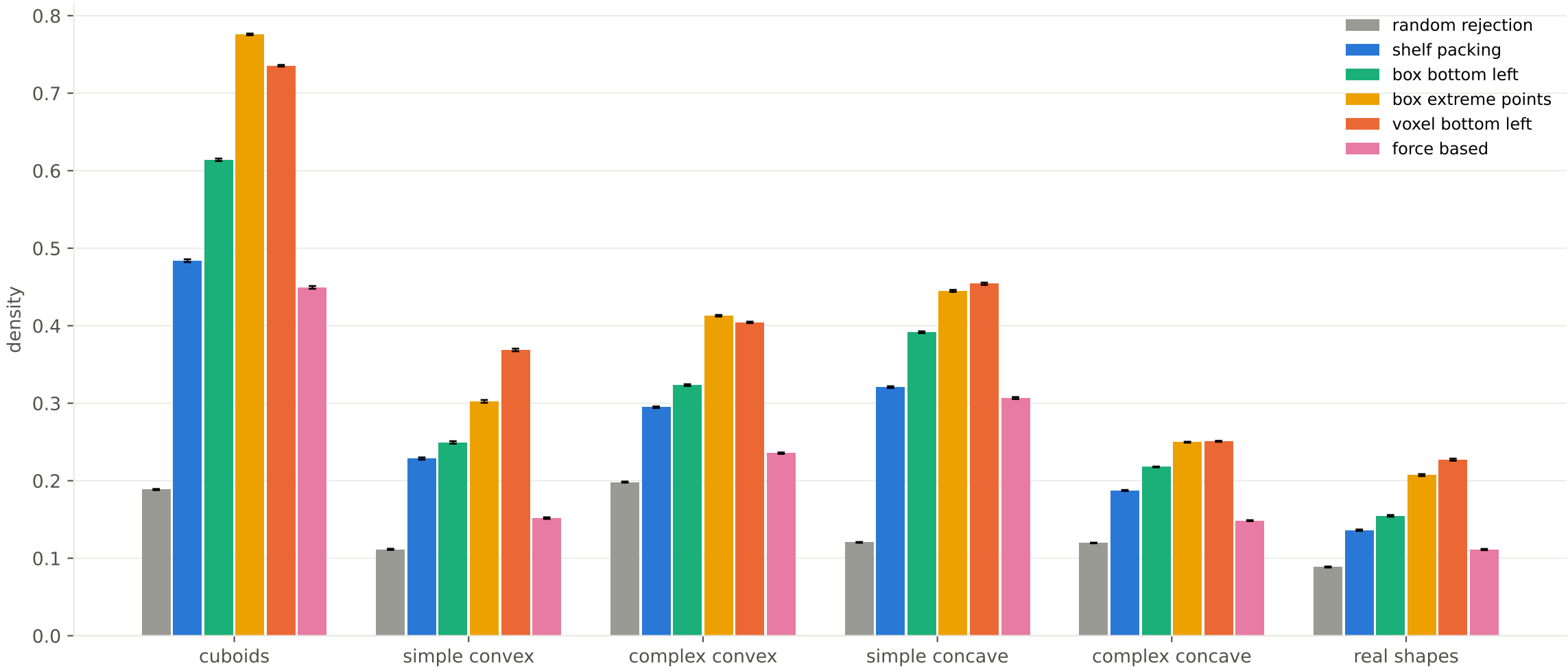
Results: cost per packing

seconds per packing on the benchmark machine (2 cores), log scale; search methods are fixed at a 5 s budget



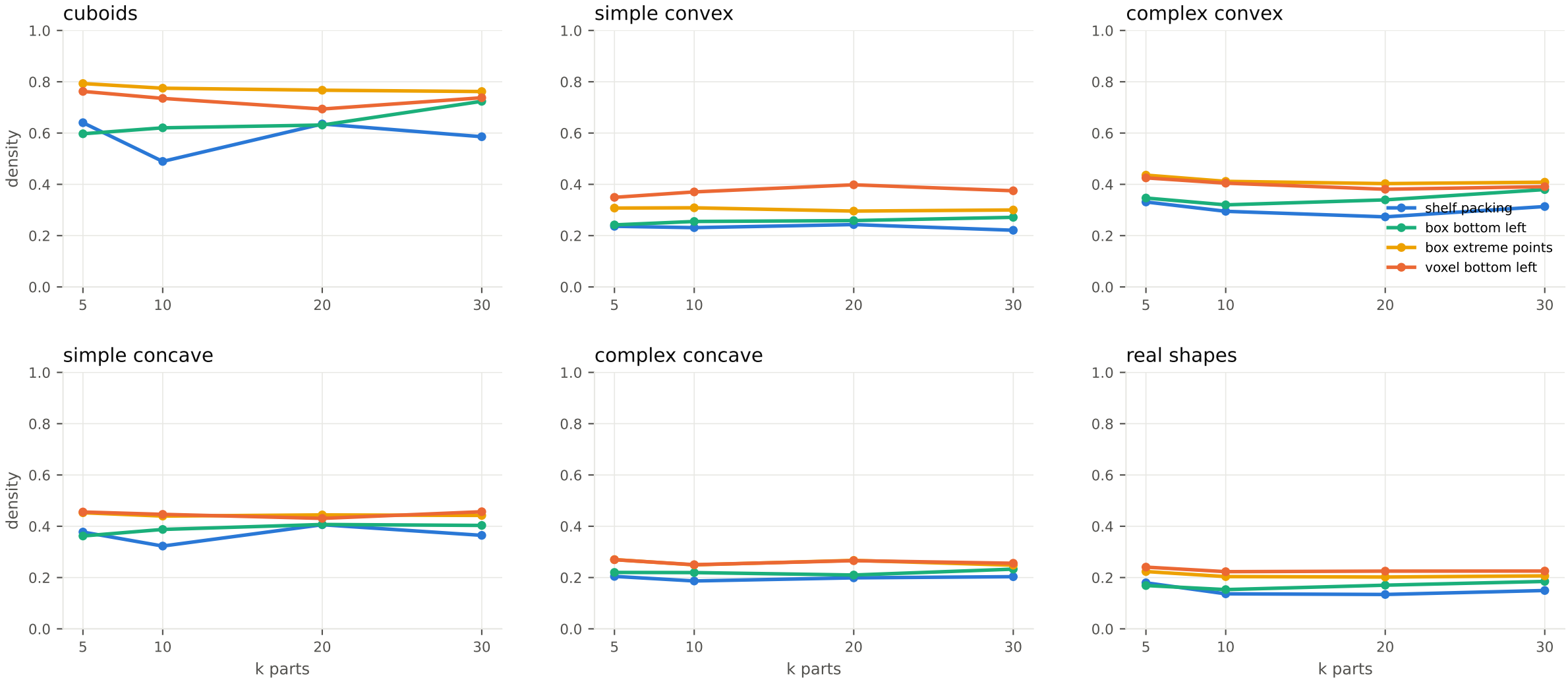
Confirmation at n = 1000

the six original methods, 36,000 packings; error bars = standard error of the mean density; same ranking as at n = 100



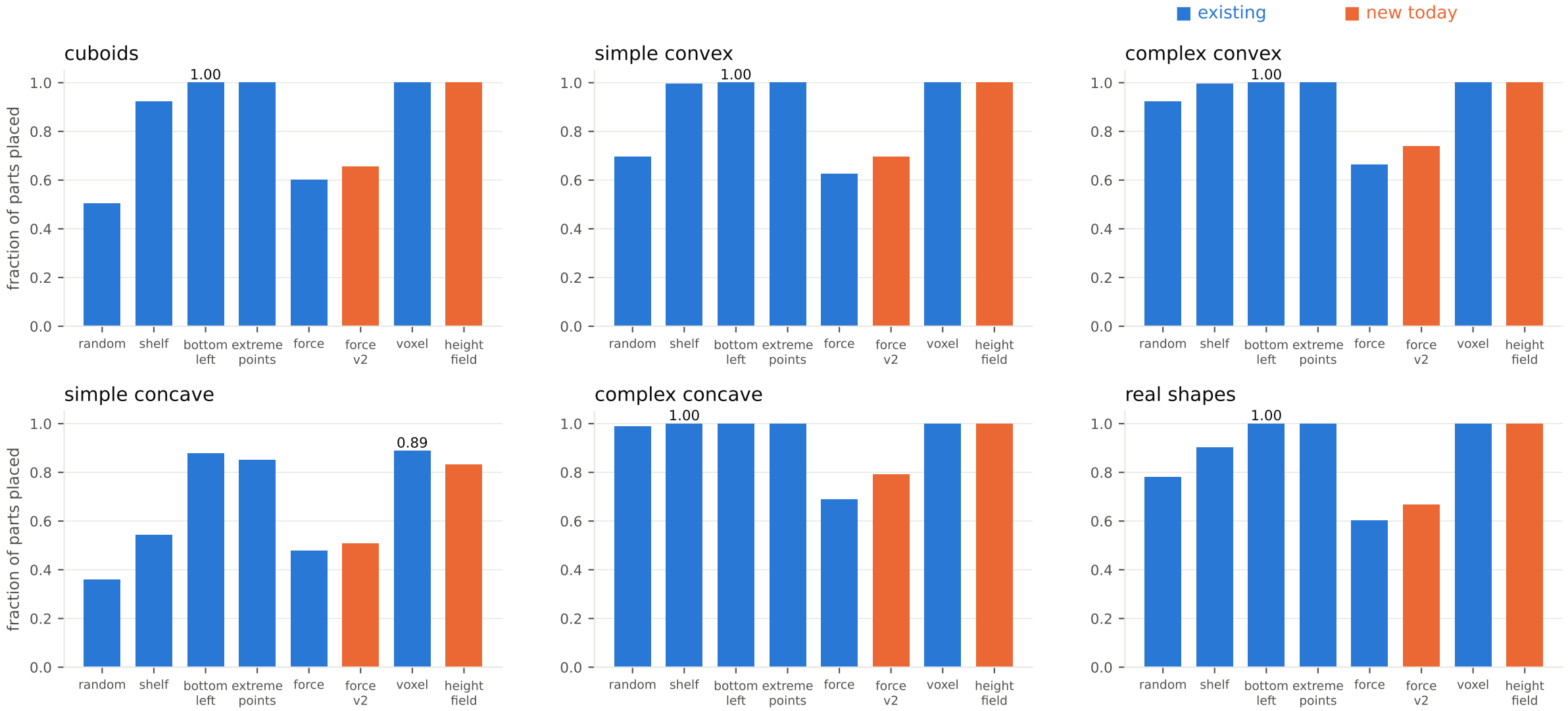
Results: density versus part count k

n = 100 per point, the same four methods: density barely depends on k from 5 to 30, so the greedy rules do not improve with more parts



Results: container mode, 100 mm cube, k = 30

share of the 30 parts that fit, n = 100; search methods not run here (5 s each); the force methods drop parts that still collide



Verification

anything checkable by code is checked by code, never judged by eye

Overlap: exact mesh booleans (manifold3d) on sample packings of every algorithm: 0 mm³ intersection

the fast voxel check flags 7 of 1,000 shape-based packings at 0.5 to 0.7% (threshold 0.5%); all 7 re-checked exactly: 0 mm³

Clearance: the minimum gap is measured from dense surface samples (0.5 mm spacing) and checked against a

brute-force KD-tree on 54 packings: identical. Every packer keeps ≥ t = 0.5 mm in every trial

Inertia: the tensor code reproduces the closed-form tensor of a solid box exactly

Reproducibility: trial t draws its parts with seed t, so every algorithm sees the same sets; the search methods

are time-budgeted, so their numbers move by about ±0.01 between machines; a clean venv with only requirements.txt runs everything

Known limits: the voxel grid only guarantees clearance along x, y, z (diagonal gaps can be slightly under t);

box methods cannot use concave space; no assemblability (insertion path) check yet; COM and inertia are

reported but not optimized

Takeaways and next steps

1. Search beats greedy: the genetic algorithm is the densest method on 5 of 6 categories (annealing is 0.01 behind), +0.02 to +0.05 over voxel bottom-left, and with the most compact boxes and the lowest COM offset of any method
2. Shape beats boxes on concave and real parts; the 2 mm grid and the 0.5 mm clearance cap the gain
3. Heightfield drop is the cheap shape-aware option: 0.05 s, overlap-free by construction, within 0.02 of voxel
4. Density costs shape in the greedy methods: 'smallest box' choices make slabs and towers (aspect 4 to 6); force v2 fixes the rows of v1 (+10 to 25% density) but is still 20 to 30% below the greedy methods
5. In a 100 mm cube with 30 parts, voxel, heightfield and bottom-left place the most parts; nothing controls COM yet

Next

multi-objective score: density, aspect ratio (or target box), COM offset; decide which are limits, which are optimized

assembly feasibility: straight-line insertion along ±x, ±y, ±z given the placement order

longer search budgets and a parallel decoder for the GA; 1 mm grid and 24 orientations for the shape-based methods

connect the viewer to the team GUI so 'pack these parts' shows the algorithms side by side

Appendix: 60-part categories, k = 10, 20, 30, 13 algorithms

n = 300 per bar for the fast methods, n = 100 for the search methods; error bars and bands are 99% confidence intervals of the mean

What changed from the main study

part pool: 60 random parts per synthetic category instead of 20 (real shapes unchanged, 23)

three more algorithms: empty maximal spaces (box), genetic heightfield (search with the heightfield decoder), best of fast (runs the six fast packers, keeps the densest)

two more measures: contacts (part pairs whose surfaces come within t + 0.5 mm) and isolated parts (parts touching nothing)

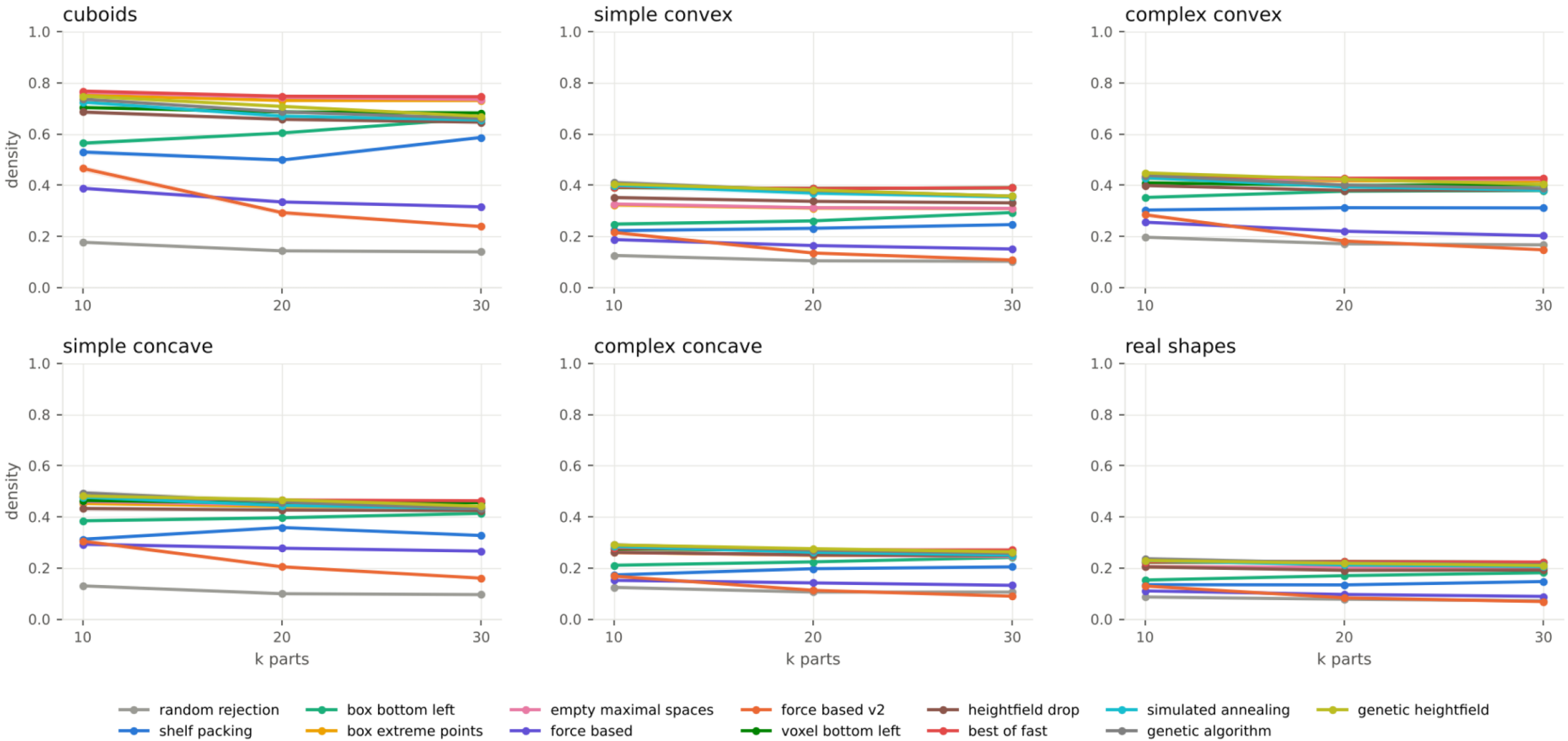
every trial's packing is saved (results/solutions_*.jsonl) and can be reopened with replay.py

Findings

1. k = 10: the genetic algorithm leads on 5 of 6 categories but by only +0.01 over best of fast; on cuboids it loses (0.748 vs 0.767)
2. k = 20 and 30: the 5 s search budget no longer covers the order space; best of fast, EMS and voxel lead on density everywhere
3. the greedy methods' towers get worse with k: aspect ratio 4 to 5 at k = 10, 7 to 9 at k = 20, 10 to 12 at k = 30; genetic heightfield holds aspect ratio about 2 at every k at a 3 to 6% density cost
4. empty maximal spaces is the best box method at every k (ties or beats extreme points, same cost)
5. contacts: box methods touch 2 to 3 neighbours per part on blocky parts; voxel and heightfield parts sit 1 to 2 mm apart (the 2 mm grid), so they register few contacts within the 1 mm band; the band should scale with the grid pitch
6. random rejection and force-based v2 fall apart as k grows (v2: 0.47 at k = 10, 0.29 at k = 20 on cuboids)

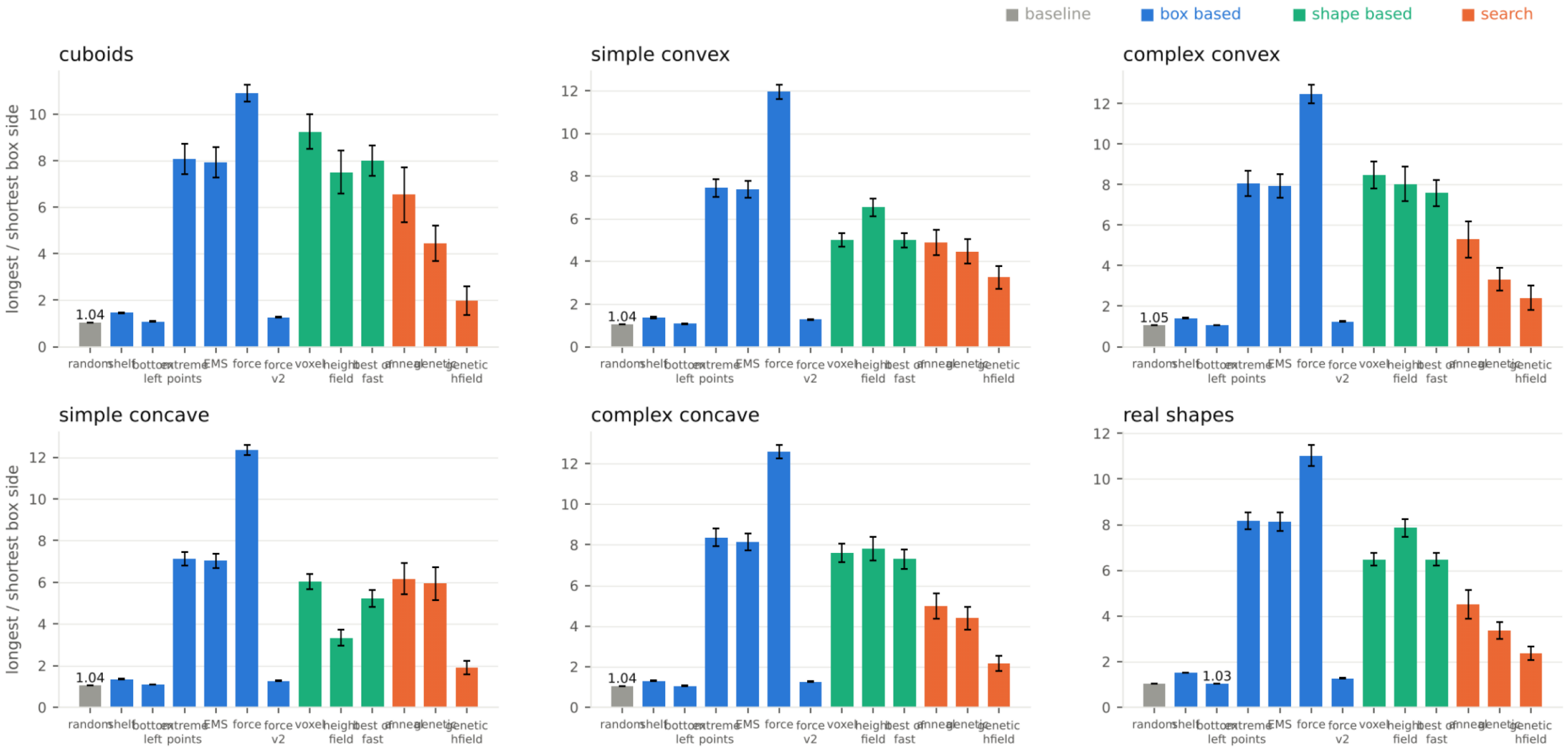
Density versus part count k

shaded band = 99% confidence interval of the mean



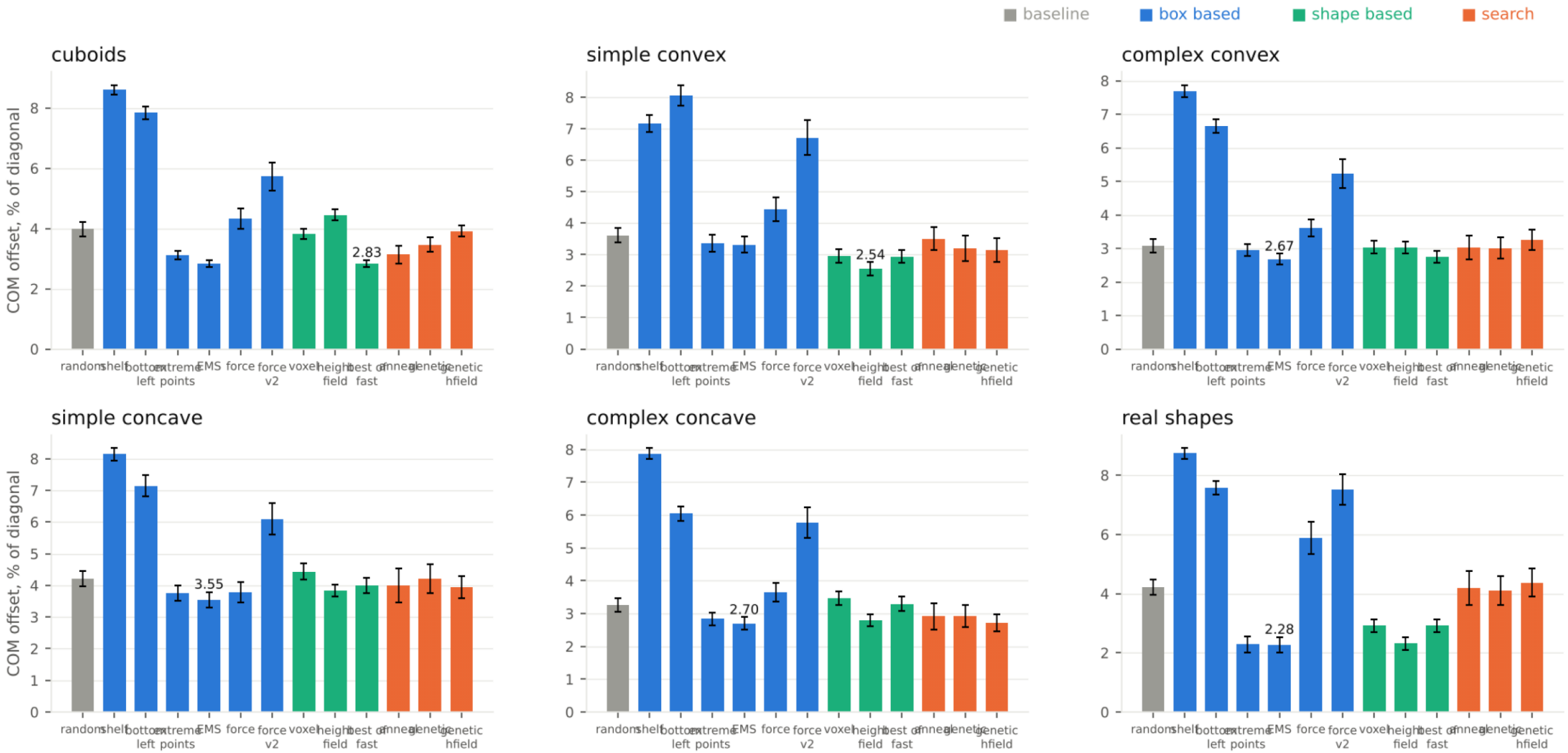
Aspect ratio (lower = more compact), k = 20

k = 20, open mode, 60-part categories (23 real shapes); error bars = 99% confidence interval of the mean; n per bar in the leaderboard CSV



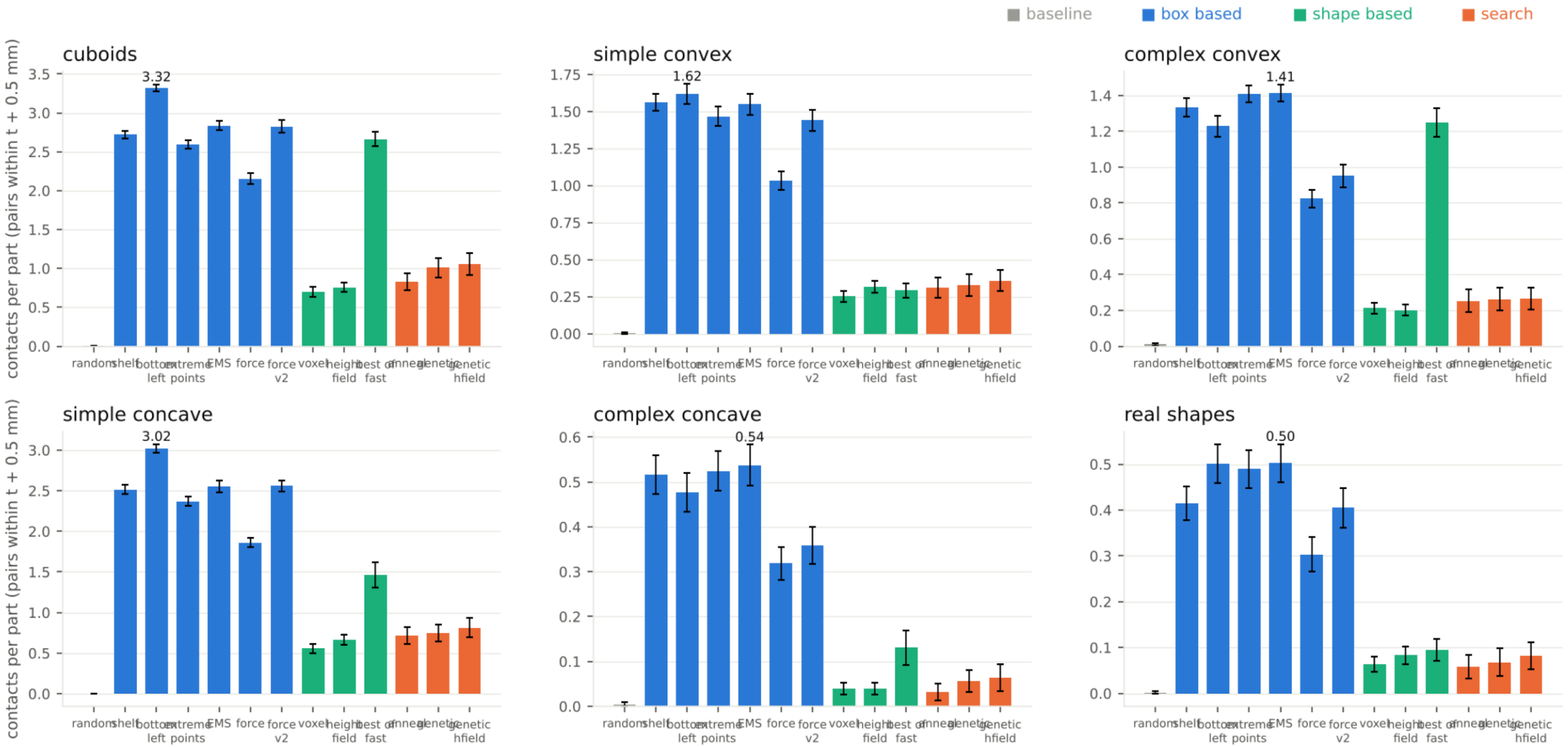
Center of mass offset (lower = better balanced), k = 20

k = 20, open mode, 60-part categories (23 real shapes); error bars = 99% confidence interval of the mean; n per bar in the leaderboard CSV



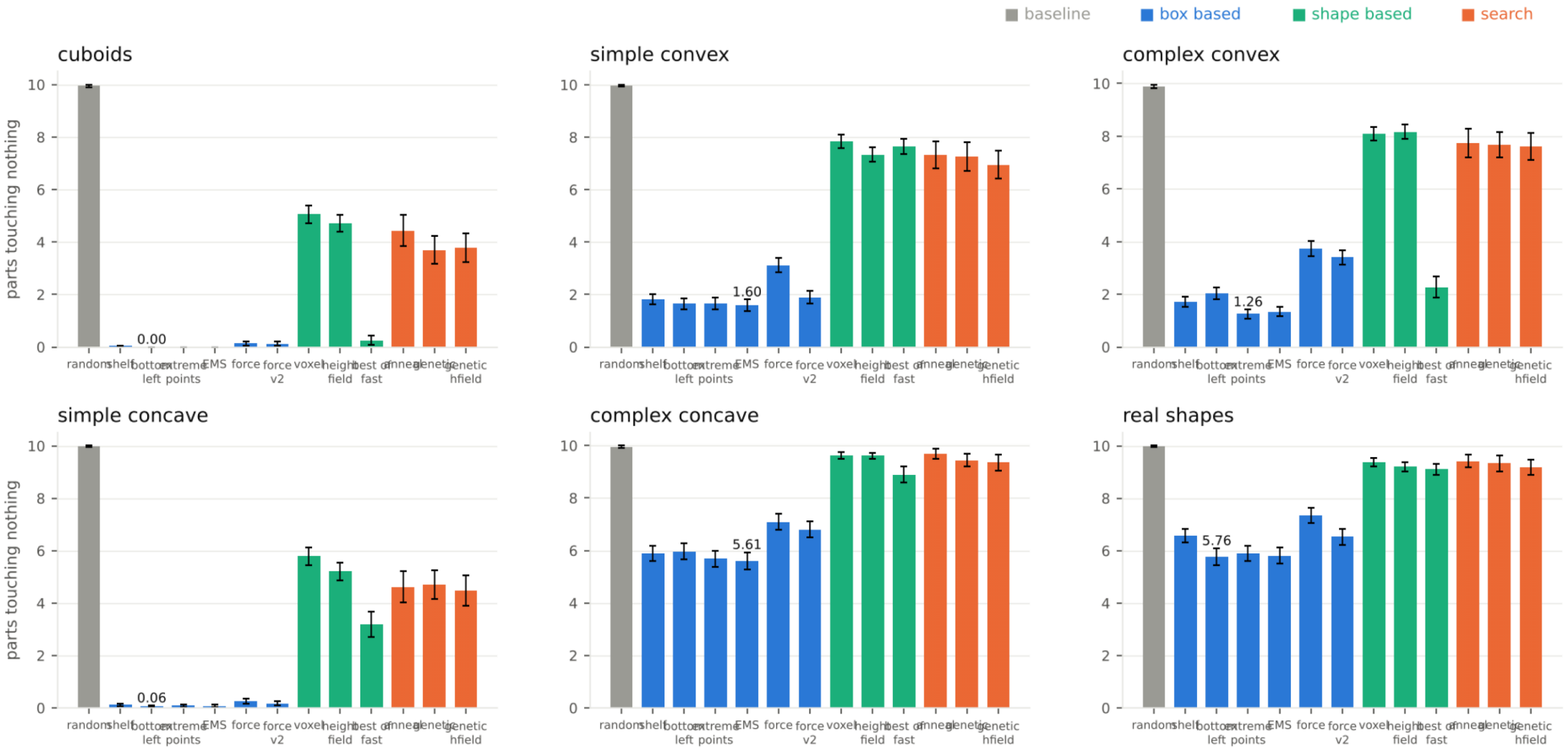
Contacts per part (higher = more parts touch through the clearance), k = 10

k = 10, open mode, 60-part categories (23 real shapes); error bars = 99% confidence interval of the mean; n per bar in the leaderboard CSV



Isolated parts (parts with no contact, i.e. nothing to support them), k = 10

k = 10, open mode, 60-part categories (23 real shapes); error bars = 99% confidence interval of the mean; n per bar in the leaderboard CSV



Cost per packing, k = 30

k = 30, open mode, 60-part categories (23 real shapes); error bars = 99% confidence interval of the mean; n per bar in the leaderboard CSV

